

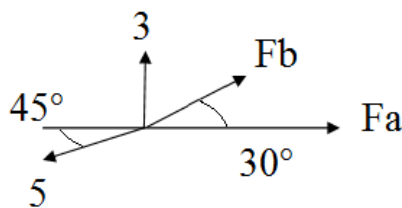
Cours 02 – Notion 1 : Introduction à la programmation orientée objet

2.1.1 Définitions

La programmation orientée objet se définit comme un paradigme de programmation qui tente de modéliser un concept complexe à l'intérieur d'une classe. Dans cette classe, on y décrit les différentes valeurs natives que possède l'objet complexe sous la forme d'attributs. De plus, on décrit dans la classe les différentes actions qui manipulent cet objet sous la forme de méthodes. Puisque plusieurs représentations d'un même objet peuvent exister, on nomme ces différentes représentations des instances.

2.1.2 Illustration des définitions

Prenons en exemple des forces physiques.



Dans cet exemple, on retrouve quatre instances de la classe force : Fa, Fb, la force de 3N et la force de 5N.

Chacune de ces forces possède plusieurs de ces attributs (notez qu'il est possible que certains attributs ne soit pas utilisée pour une instance) :

- Un nom
- Un module
- Une orientation
- Un point d'action
- Est-ce une inconnue dans le système?
- Un coefficient d'équation

Visualisation des quatre instances séparément :

nom	: Fa
module	: ?
orientation	: 0
positionX	: 0
positionY	: 0
estConnue	: 0
coefficient	: 1

nom	: Fb
module	: ?
orientation	: 30
positionX	: 0
positionY	: 0
estConnue	: 0
coefficient	: 1

nom	: ''
module	: 3
orientation	: 90
positionX	: 0
positionY	: 0
estConnue	: 1
coefficient	: 1

nom	: ''
module	: 5
orientation	: 225
positionX	: 0
positionY	: 0
estConnue	: 1
coefficient	: 1

Il serait possible d'accomplir plusieurs actions sur ces forces, ces actions représenteraient les méthodes :

- Additionner une force à une autre
- Définir une nouvelle force
- Déplacer une force
- Comparer deux forces entre elle

2.1.3 Organisation d'une classe

Dans la définition d'une classe, on retrouve la description de tous ces attributs de même que la description de toutes ces méthodes. Ainsi, plusieurs fonctions seront codées à l'intérieur d'un même fichier. Le nom du fichier doit donc également la classe.

```
public class Force
{
    /** *****
    /* DÉFINITIONS DES ATTRIBUTS PUBLIQUES
    /** *****
    public String nom;           //Le nom de la force pour la reconnaître
                                //dans un système d'équation.

    /** *****
    /* DÉFINITIONS DES ATTRIBUTS PRIVÉES
    /** *****
    private double positionX;    // Le point ou agit la force.
    private double positionY;

    private double module;      // La norme du vecteur force.
    private double orientation; // L'orientation du vecteur force.

    private boolean estConnue;   // 1 si le module et l'orientation
                                // sont connus

    private double coefficient;  // Le coefficient de la force dans un
                                // système d'inconnu si le module
                                // n'est pas connu.

    /** *****
    /* DÉFINITIONS DES MÉTHODES PUBLIQUES
    /** *****
    public Force(String nom)
    {
        //Crée une nouvelle Force nommée.
        //On la considère comme une inconnue dans un système d'équation.
    }

    public Force(String nom, double module, double orientation)
    {
        //Crée une Force nommée, ayant le module et l'orientation reçue
        //On la considère comme un élément connu dans un syst. d'équation.
    }

    public Force Copie()
    {
        //Crée une copie de cette la Force actuelle.
    }

    public String toString()
    {
        //Crée une chaîne de caract. contenant les attributs de la force
    }

    public boolean estEgale(Force autre)
    {
        //Compare la force actuelle avec l'autre Force reçue.
    }

    /** *****
    /* DÉFINITIONS DES MÉTHODES PRIVÉES
    /** *****
    private boolean estMomentAppuis()
    {
        //Détermine si la force actuelle crée une moment de force sur
        //appui d'un treillis.
    }
}
```

- L'instruction "`public class`" permet de déclarer la classe.
- L'utilisation du terme "`public`" ou "`private`" permet de spécifier la visibilité d'une propriété de la force. Seules les instructions de la classe Force.java pourront accéder en écriture ou en lecture aux propriétés privées de la classe Force, alors qu'une instruction placée dans n'importe quel fichier (Application.java, Treillis.java) pourra fixer ou modifier le nom d'une force puisque cet attribut est public.
- L'utilisation du terme "`public`" ou "`private`" permet de spécifier la visibilité d'une méthode de la force. Seules les instructions de la classe Force.java pourront utiliser la méthode estMomentAppuis puisque cette méthode est privée, alors que les cinq autres méthodes pourront être appelées dans n'importe quel fichier (Application.java, Treillis.java) puisqu'elles sont de nature publique.